Niharika Pawar
D12C Roll no.44

TCS Self Learning
JFLAP

1) Deterministic Finite Automata

Theory:

A Deterministic Finite Automaton (DFA) is a computational model used to identify whether an input string belongs to a particular regular language. It consists of a finite number of states connected through transitions. The machine processes an input string one symbol at a time and moves between states according to predefined transition rules.

In a DFA, every state has exactly one transition for each symbol in the input alphabet. Therefore, the next state is always uniquely determined. After processing the entire string, the input is accepted if the machine reaches a final state; otherwise, it is rejected.

Formal Definition:

A DFA is represented by a five-tuple:
$M = (Q, \Sigma, \delta, q_0, F)$
Where:

- Q = Finite collection of states.
- Σ = Set of input symbols.
- δ = Transition function, $\delta: Q \times \Sigma \rightarrow Q$.
- q_0 = Initial state of the automaton.
- F = Set of final or accepting states.

Working Principle:

The automaton begins at its initial state and reads each character of the input sequentially. Based on the current state and input symbol, it follows the corresponding transition. This process continues until the complete string has been processed.

For example, a DFA can be designed to recognize all strings over the alphabet that begin and end with 'c'. The machine tracks the first character and updates its state according to subsequent symbols. Only strings satisfying both conditions are accepted.

DFAs are commonly used in lexical analysis, pattern recognition, text processing, and compiler design.

Example:

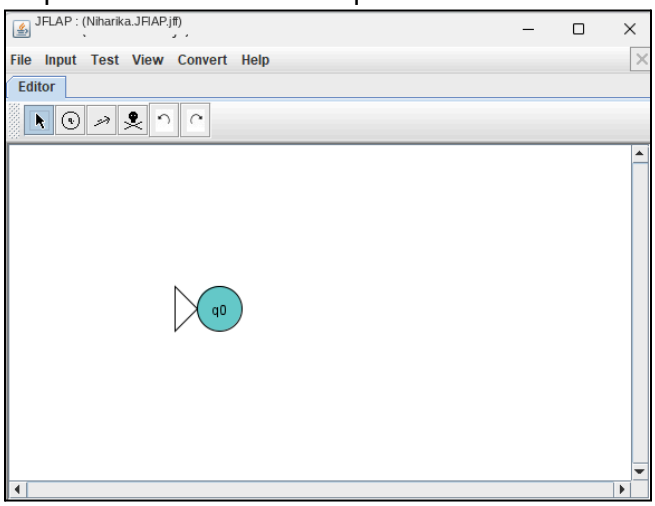
Consider the regular language L over the alphabet { a, b, c } comprised of all strings that begin and end with c. Let's construct a DFA M to recognize that language.

Some strings in the language include: c, cc, ccc, cac, cabc, cabcbabc
Some strings not in the language include: λ, a, ac, cb, ccca

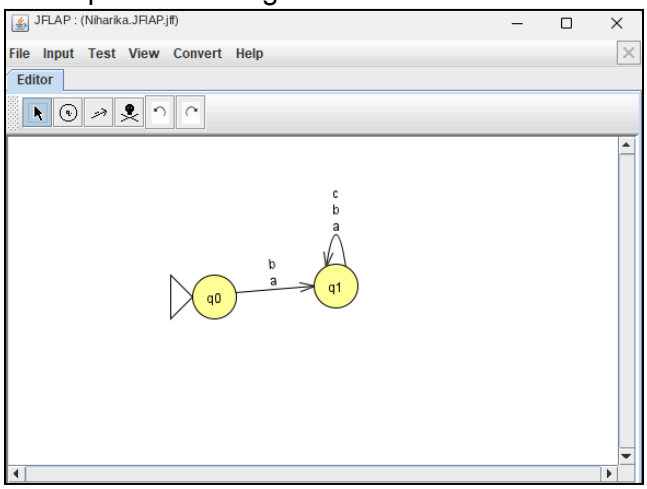
Niharika Pawar
D12C Roll no.44

Execution:

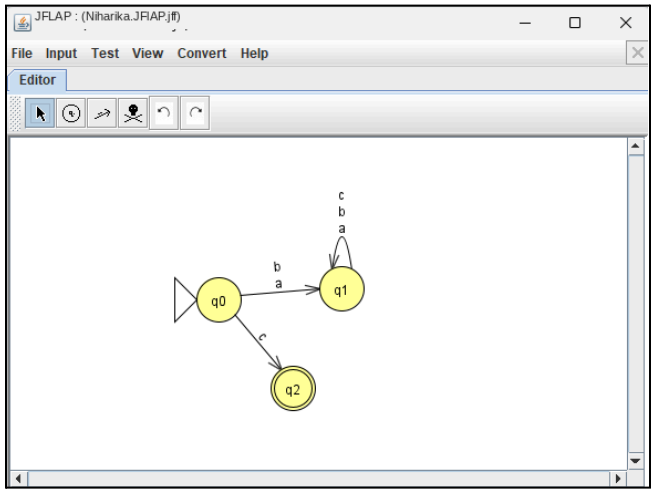
Step 1: Create initial state q0



Step 2: Create a new state, q1, as the destination of a or b from q0 and which traps all subsequent substrings

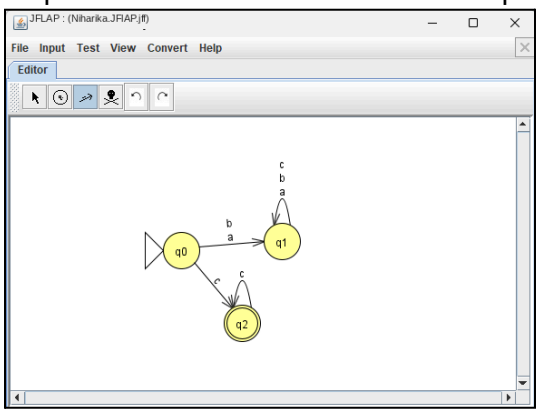


Step 3: Create a new state, q2, as the destination of transition c from q0 and which is an accept state

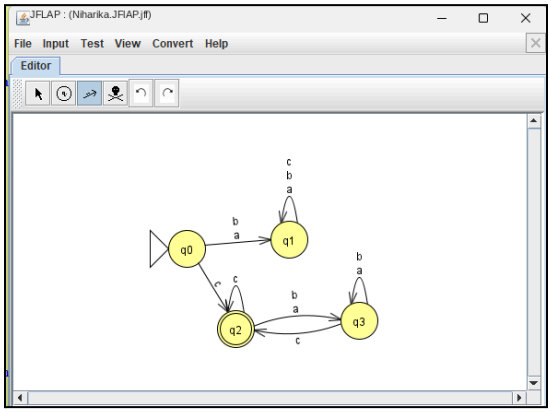


Niharika Pawar
D12C Roll no.44

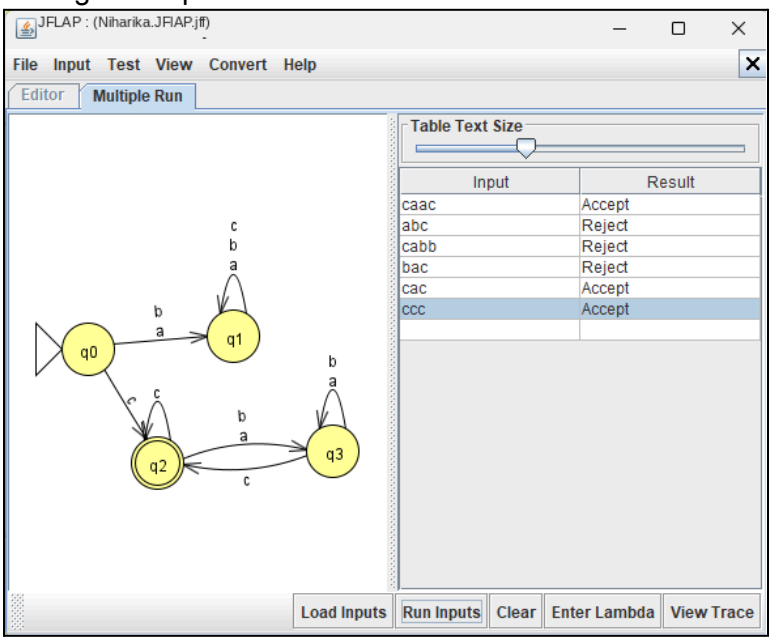
Step 4: Add a c transition to and from q2



Step 5: Add state q3 with a transition of a or b from q2. Add transitions to remain in q3 for a or b. Add transition from q3 back to q2 for c



Testing with inputs:



Therefore the DFA accepts all strings that start and end with c

2) Non Deterministic Finite Automata

Theory:

A Non-Deterministic Finite Automaton (NFA) is a finite-state machine in which multiple transitions may be possible for the same input symbol from a particular state. Unlike a DFA, an NFA does not require exactly one transition for every input symbol.

An NFA may move to different states for the same input or may have no transition for certain symbols. It can also support epsilon (ε) transitions, allowing movement between states without consuming an input character.

A string is considered accepted when at least one possible execution path leads to an accepting state after processing the complete input.

Formal Definition:

An NFA is expressed as:
$M = (Q, \Sigma, \delta, q_0, F)$
Where:

- Q = Finite set of states.
- Σ = Input alphabet.
- δ = Transition function, $\delta: Q \times (\Sigma \cup) \rightarrow P(Q)$.
- q_0 = Starting state.
- F = Collection of accepting states.

Working Principle:

The NFA starts from its initial state and explores possible transitions for each input symbol. When multiple transitions are available, the automaton considers different computational paths. Acceptance depends on whether any one of these paths successfully reaches a final state.

For example, an NFA can recognize strings over that end with 'aa'. The automaton allows any sequence of a's and b's before identifying the final two consecutive a's.

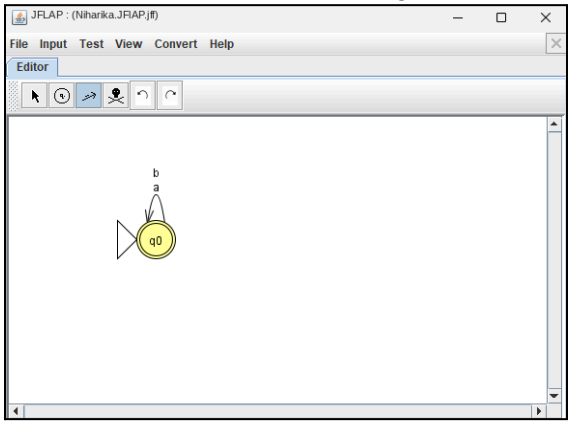
Although NFAs and DFAs operate differently, both recognize the same class of languages known as regular languages.

Example:

Define an NFA that recognizes the following language L over Σ = {a, b}:
L = { w | w ends with aa }.

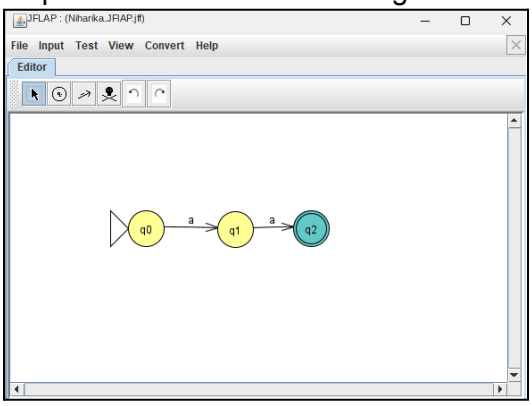
Execution:

Step 1: Build an NFA that recognizes the set of all strings {a, b}*

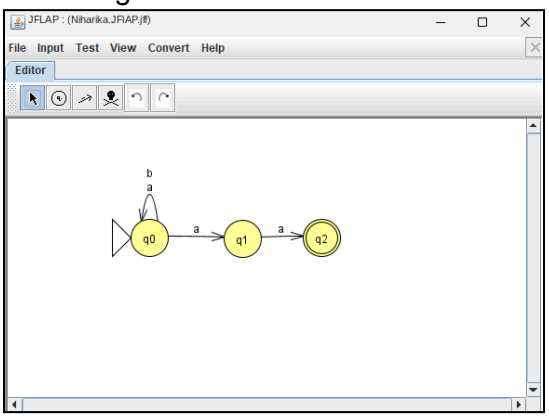


Niharika Pawar
D12C Roll no.44

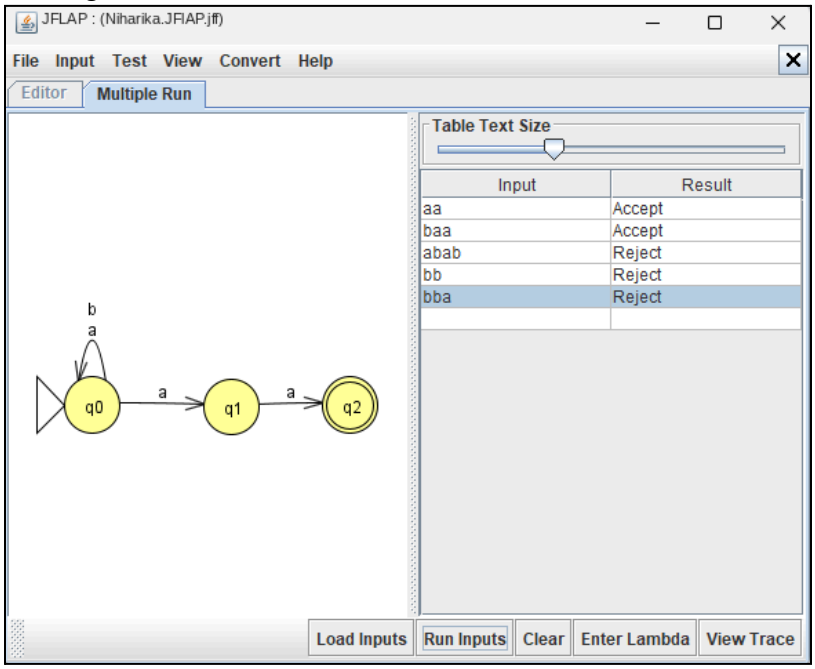
Step 2: Build an NFA that recognizes the string aa, such as the following



Step 3: Create an NFA that recognizes the sequence of these strings (their concatenation) by combining the final state of the first NFA with the initial state of the second NFA.



Testing:



Therefore, this NFA successfully accepts all strings ending with aa.

Niharika Pawar
D12C Roll no.44

3) NFA to DFA Conversion

Theory:

An NFA can be transformed into an equivalent DFA without modifying the language it recognizes. This conversion is performed using a technique called Subset Construction or the Powerset Method.

Since an NFA may enter multiple states for a single input, the equivalent DFA represents these possible states together as a single combined state. Consequently, each state in the DFA corresponds to a subset of states from the original NFA.

Mathematical Representation:

For an NFA:
$N = (Q, \Sigma, \delta, q_0, F)$
The equivalent DFA is:
$D = (Q', \Sigma, \delta', q_0', F')$
Where:

- Q' = Set of subsets of Q.
- Σ = Original input alphabet.
- δ' = Deterministic transition function.
- q_0' = Initial subset containing the NFA's start state, including its epsilon closure where applicable.
- F' = All subsets containing at least one accepting state of the NFA.

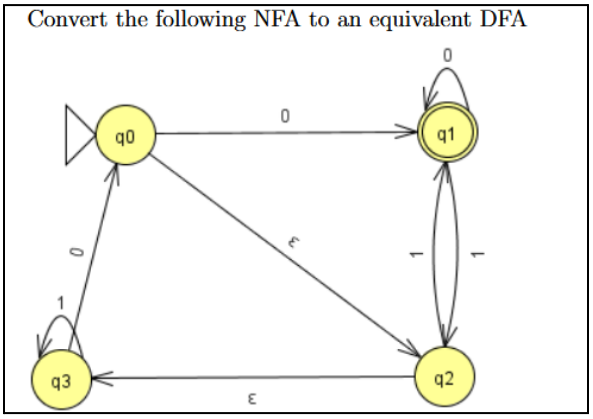
Conversion Procedure:

1. Identify the initial state of the NFA and calculate its epsilon closure if required.
2. Treat the resulting set of NFA states as the initial DFA state.
3. Determine all possible transitions for each symbol in the alphabet.
4. Combine the reachable NFA states into new DFA states.
5. Continue expanding newly generated states until no additional subsets are obtained.
6. Mark every DFA state containing an NFA final state as accepting.

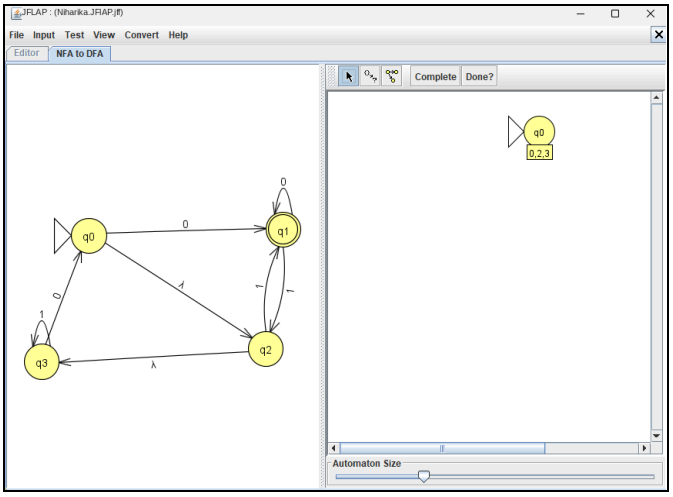
The constructed DFA processes each input through a single deterministic path while producing the same acceptance results as the original NFA.

This conversion demonstrates that nondeterminism does not increase the language-recognition capability of finite automata.

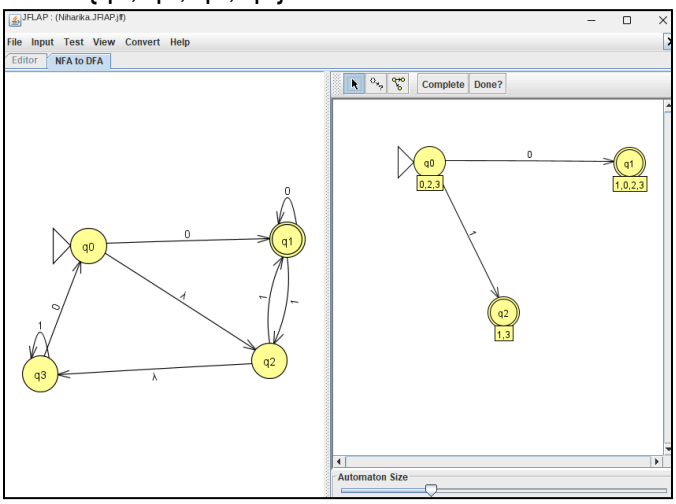
Example:



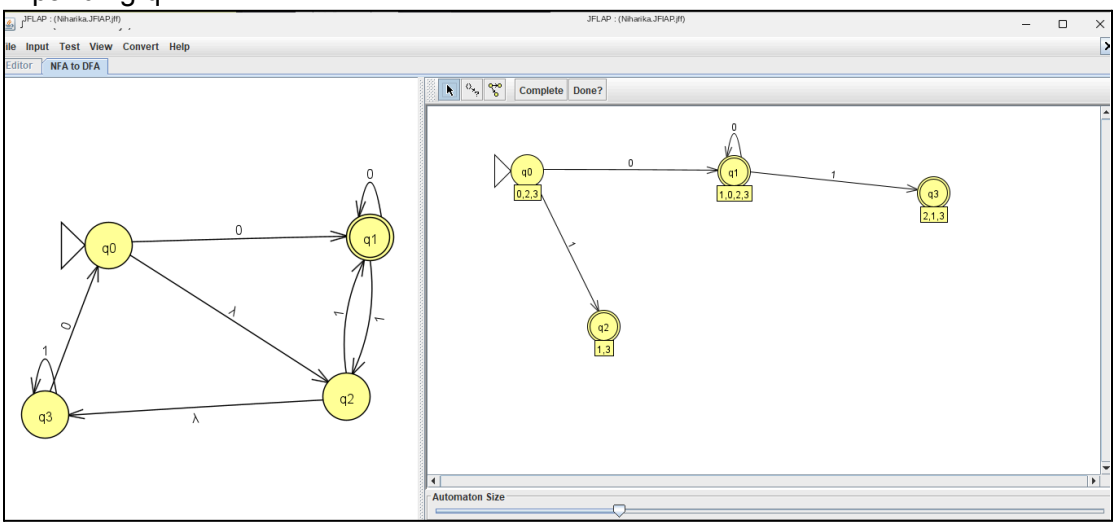
Niharika Pawar
D12C Roll no.44



In the NFA, q0 goes to q1. q2 goes nowhere on a 0. And finally q3 goes to q0 on a 0. So currently we have the subset of states {q0, q1}. But due to transitions the state will be {q0, q1, q2, q3}

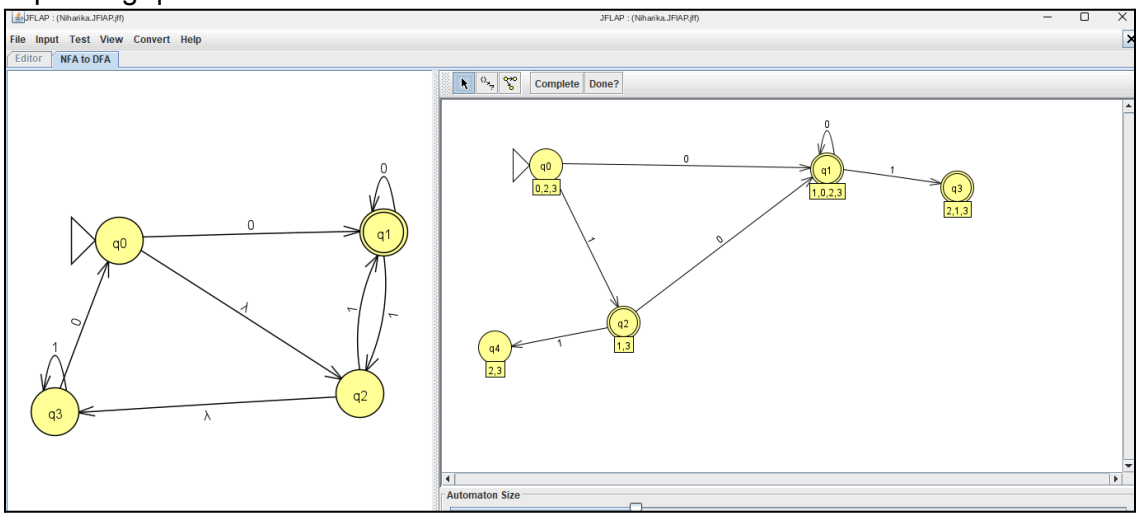


Expanding q1

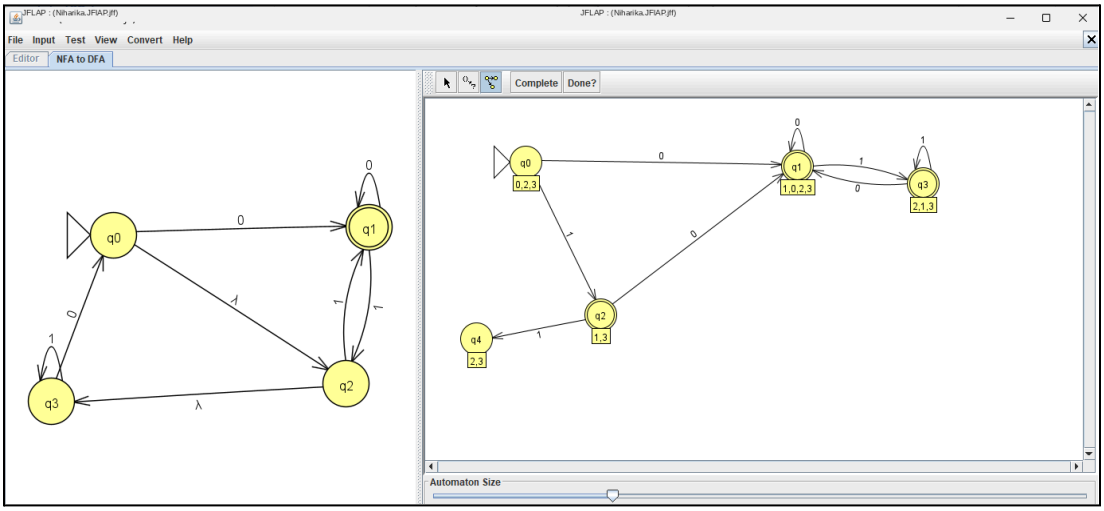


Niharika Pawar
D12C Roll no.44

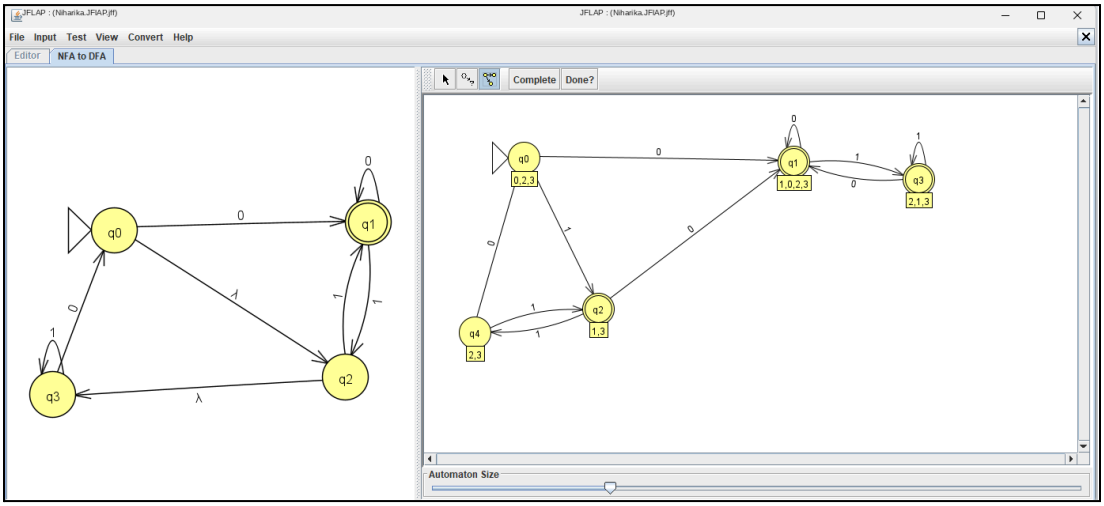
Expanding q2

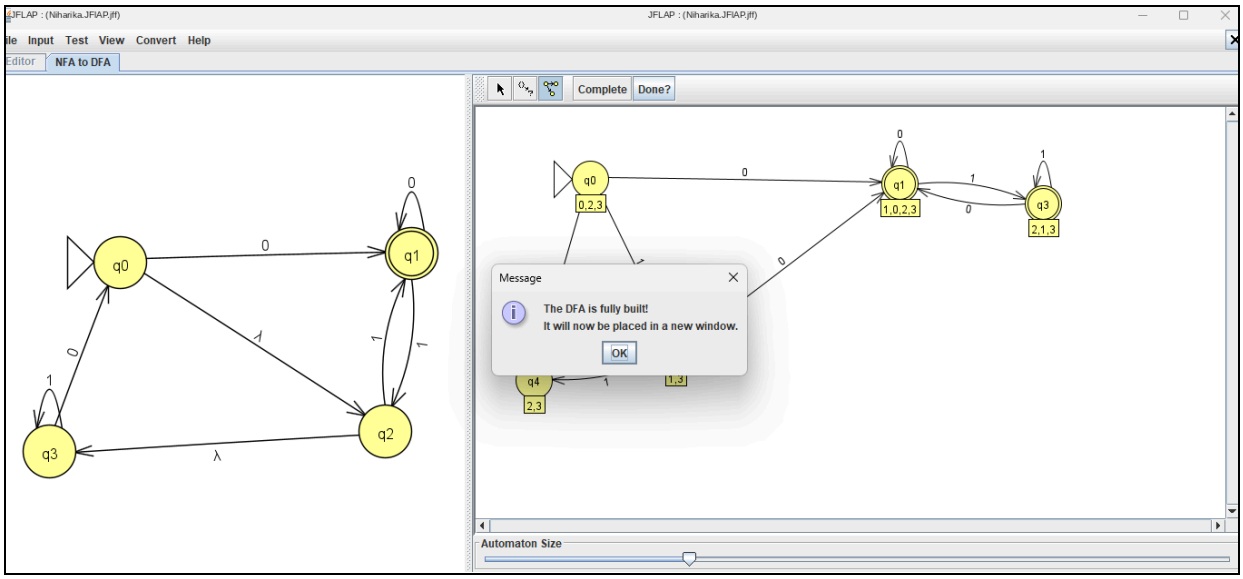


Expanding q3

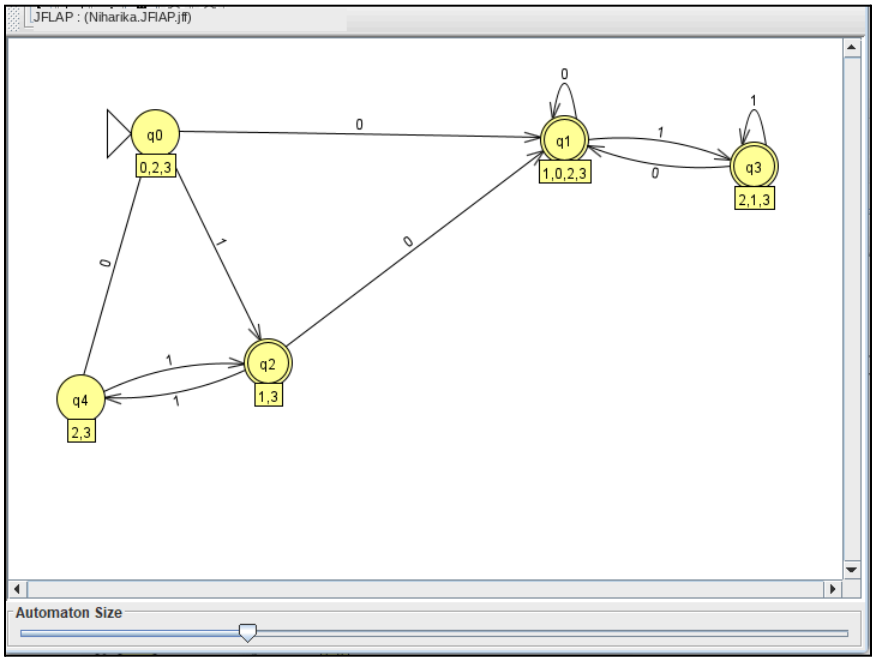


Expanding q4





Final DFA



4) DFA Minimization

Theory:

DFA Minimization is the process of reducing the number of states in a deterministic finite automaton without changing the language accepted by it. The resulting automaton is called a Minimal DFA.

During minimization, unnecessary states are removed and equivalent states are combined. Two states are considered equivalent when they produce the same acceptance or rejection result for every possible input string.

The minimized automaton retains the behavior of the original DFA while using the smallest possible number of states.

Niharika Pawar
D12C Roll no.44

Concept of State Equivalence:

Two states p and q are equivalent if no input string can distinguish their behavior. If there exists at least one string that causes acceptance from one state and rejection from the other, the states are considered distinguishable and cannot be merged.

Minimization Procedure:

1. Identify and eliminate states that cannot be reached from the initial state.
2. Divide the remaining states into accepting and non-accepting groups.
3. Examine transitions from states belonging to the same group.
4. Separate states whenever their transitions lead to different partitions.
5. Repeat the partitioning process until no further division is possible.
6. Combine states belonging to the same final partition to obtain the minimized DFA.

Importance:

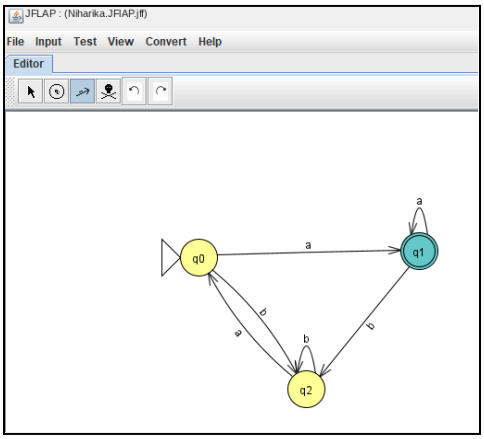
DFA minimization reduces the complexity of automata, simplifies their graphical representation, and can improve implementation efficiency by reducing the number of states that need to be maintained.
For every regular language, the minimal DFA is unique up to the renaming of states.

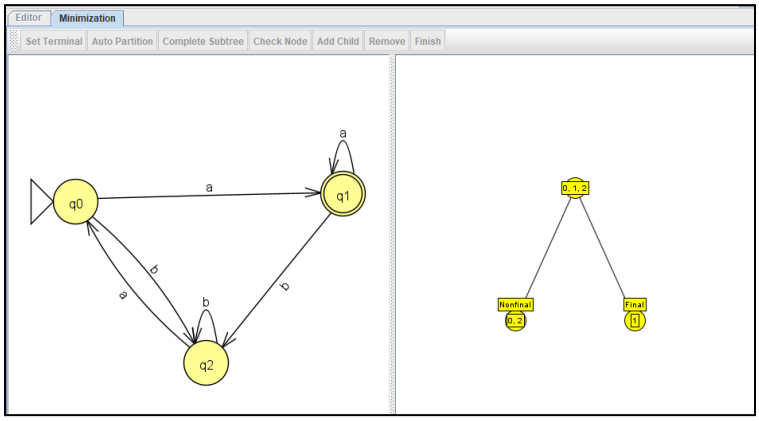
Example:

Consider the language L1 = {w ∈ Σ*|w is λ or w ends in a}.

Execution:

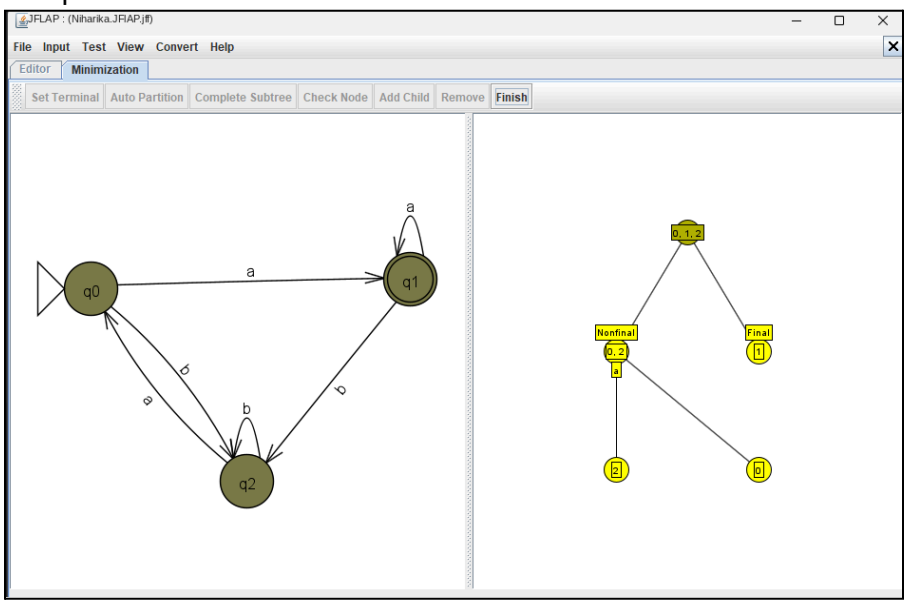
DFA:



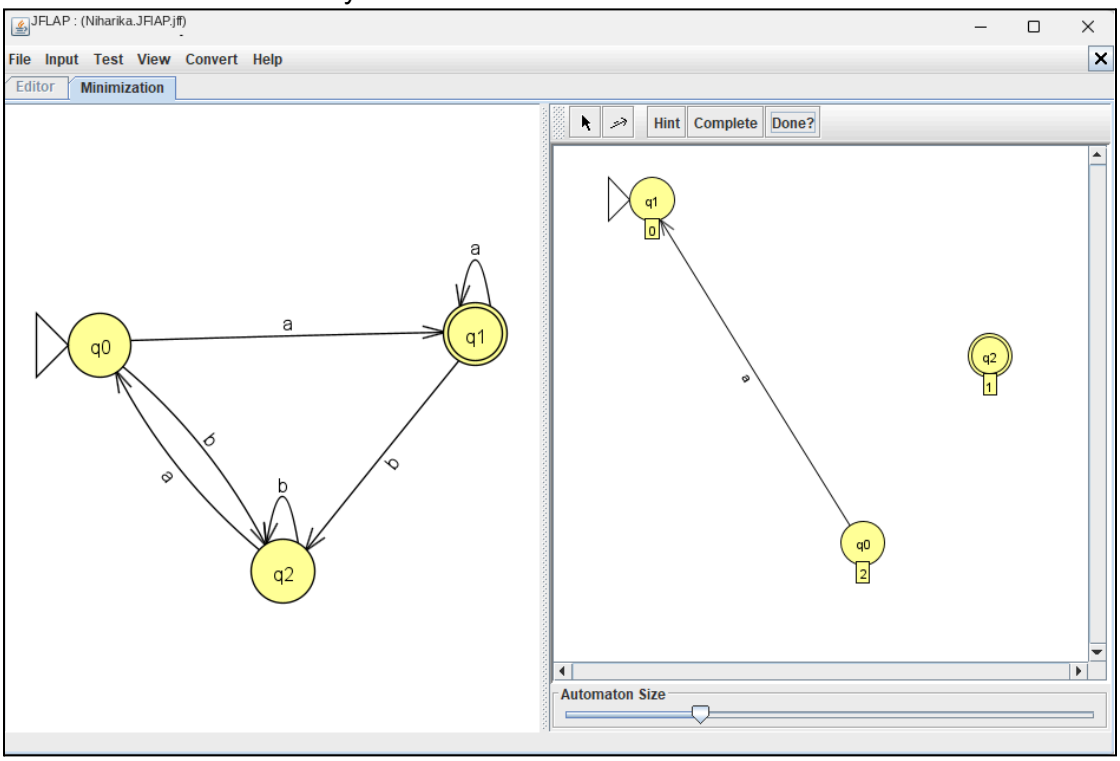


Niharika Pawar
D12C Roll no.44

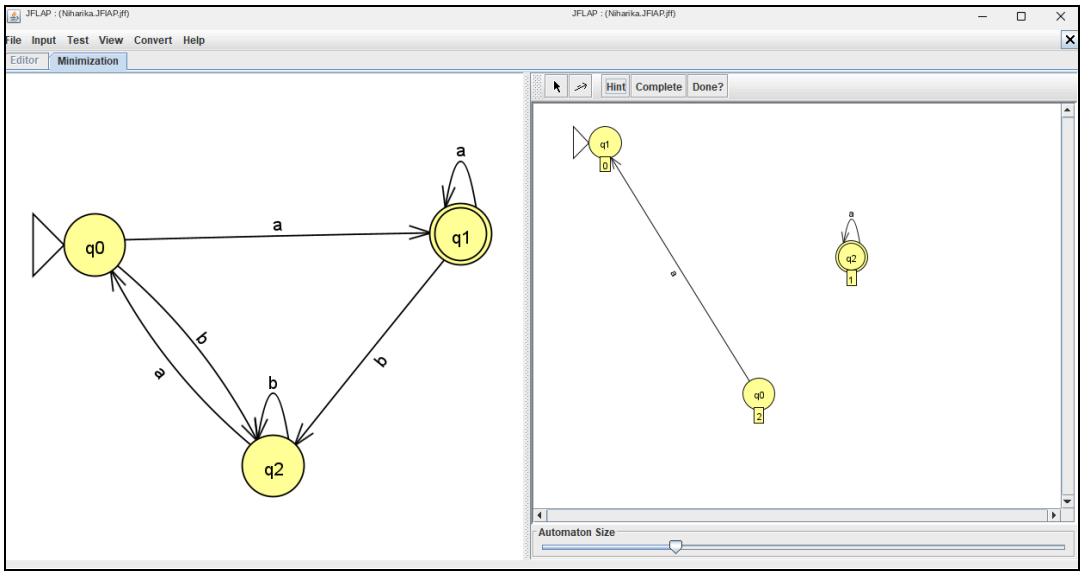
Complete Subtree

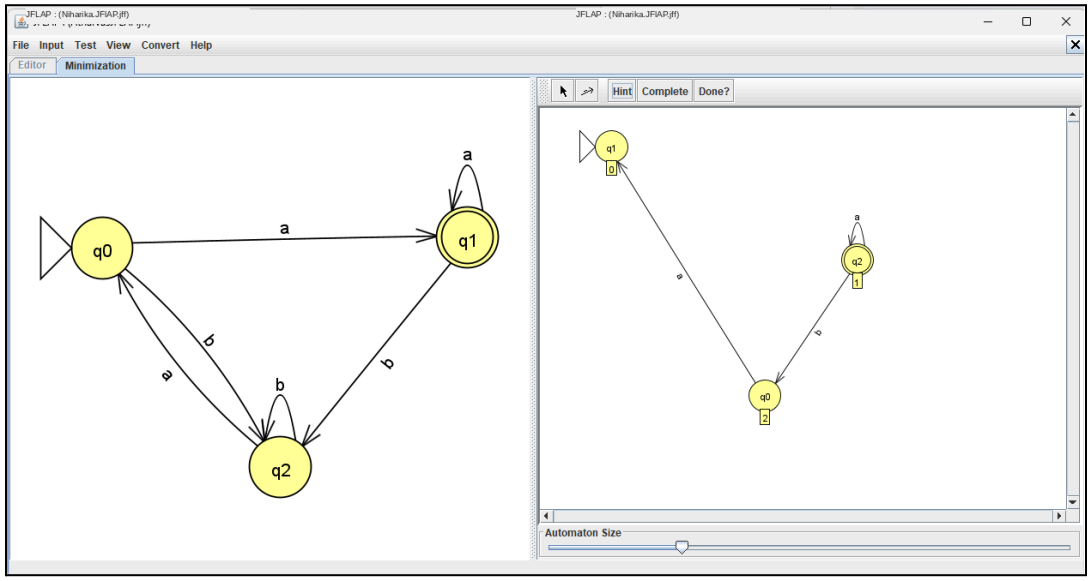


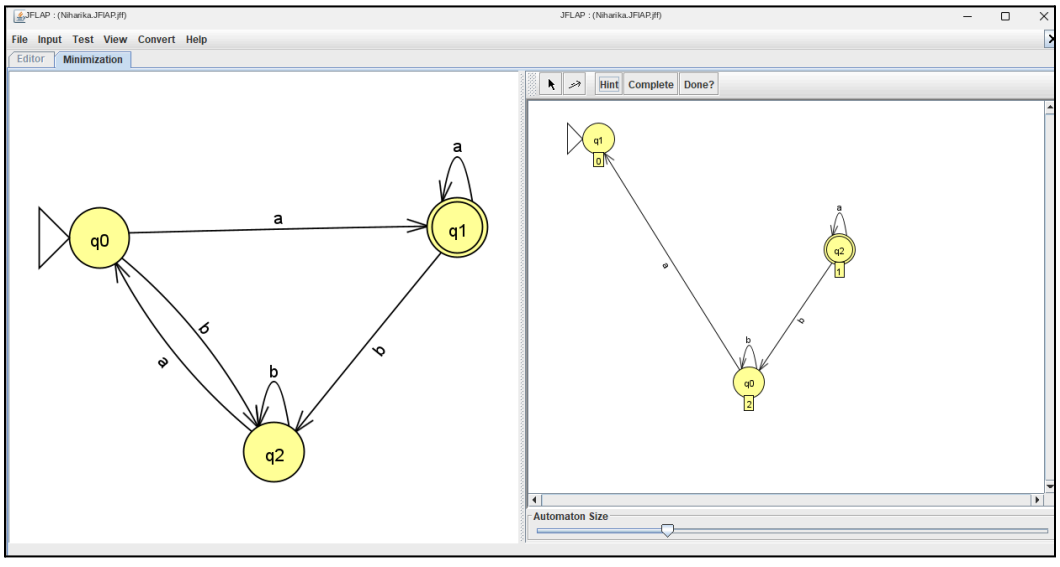
Place the transitions one by one



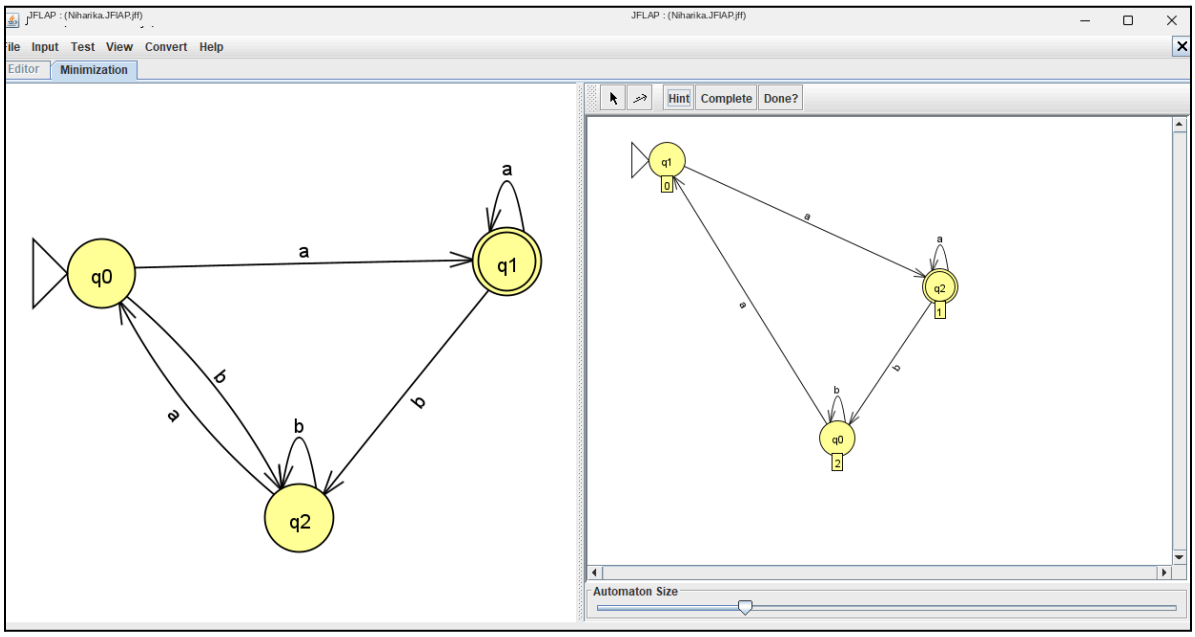
Niharika Pawar
D12C Roll no.44



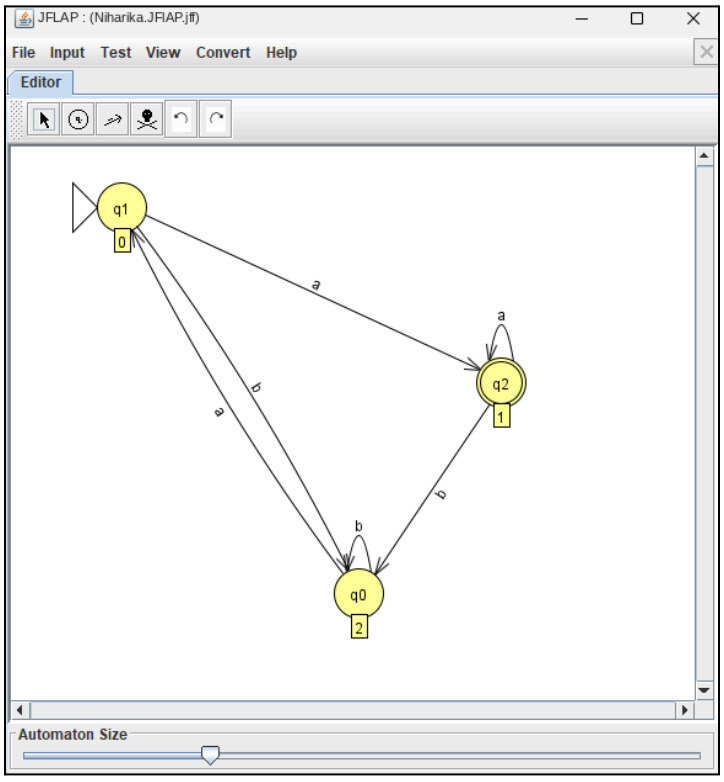




Niharika Pawar
D12C Roll no.44



Minimized DFA:



Niharika Pawar
D12C Roll no.44

5) DFA to Regular Grammar Conversion

Theory:

A Deterministic Finite Automaton can be converted into an equivalent Regular Grammar because both are capable of representing regular languages.

A regular grammar consists of production rules that generate valid strings belonging to a language. In this conversion, each state of the DFA is represented by a non-terminal symbol, while transitions are expressed as grammar productions.

A right-linear grammar generally contains production rules of the following forms:
$A \rightarrow aB$
$A \rightarrow a$
$A \rightarrow \varepsilon$
Here, A and B represent non-terminal symbols, a represents a terminal symbol, and ε represents the empty string.

Formal Representation:

A DFA is represented as:
$M = (Q, \Sigma, \delta, q_0, F)$
The corresponding regular grammar is:
$G = (V, \Sigma, P, S)$
Where:

- V = Set of non-terminal symbols corresponding to DFA states.
- Σ = Set of terminals taken from the DFA input alphabet.
- P = Collection of production rules.
- S = Starting non-terminal corresponding to the initial state.

Conversion Procedure:

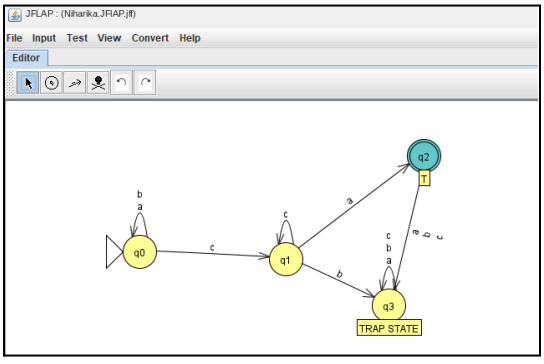
1. Assign a non-terminal symbol to each DFA state.
2. Select the non-terminal corresponding to the initial state as the grammar's start symbol.
3. For every transition $\delta(q_i, a) = q$□, introduce a production $q_i \rightarrow aq$□.
4. Add an epsilon production $q_i \rightarrow \varepsilon$ for each accepting state.
5. Combine the production rules to form the final regular grammar.

In this way, the sequence of transitions followed by the DFA is represented through grammar derivations.

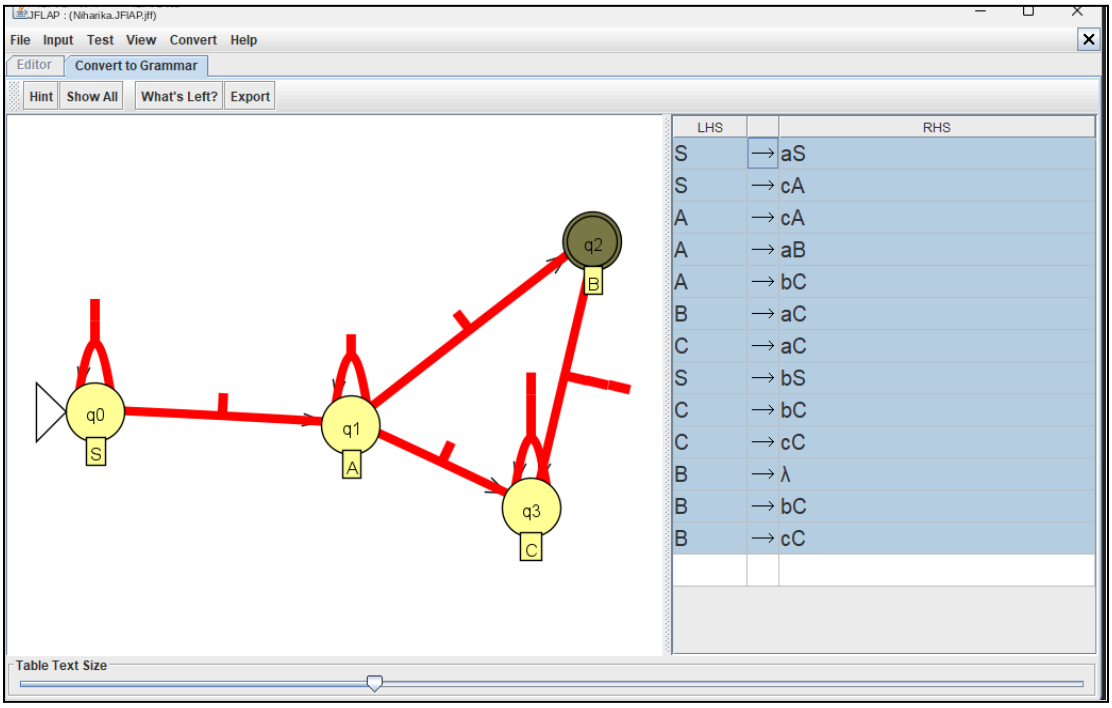
This conversion establishes the relationship between finite automata and formal grammars and demonstrates that they can describe identical regular languages.

Example:

Consider this DFA



State q0 labeled R we have the following (→ is pronounced reduces to)
R → cS
R → aR
R → bR
S → aT
S → cS
T → null



Therefore DFA has been converted to Grammer

6) DFA to Regular Expression Conversion

Theory:

A DFA can be converted into a Regular Expression (RE) that represents exactly the same set of accepted strings. Regular expressions describe patterns using operators such as union, concatenation, and Kleene star.

One of the commonly used techniques for this conversion is the State Elimination Method.

The method gradually removes intermediate states while replacing their transitions with equivalent regular expressions. The resulting expression preserves all possible accepted paths from the initial state to the final state.

Basic Regular Expression Operators:

- Union (a + b): Represents a choice between a and b.
- Concatenation (ab): Represents a followed by b.
- Kleene Star (a*): Represents zero or more repetitions of a.
- Epsilon (ε): Represents an empty string.

Conversion Procedure:

1. Construct or open the DFA to be converted.
2. Introduce a new initial state connected to the original starting state through an epsilon

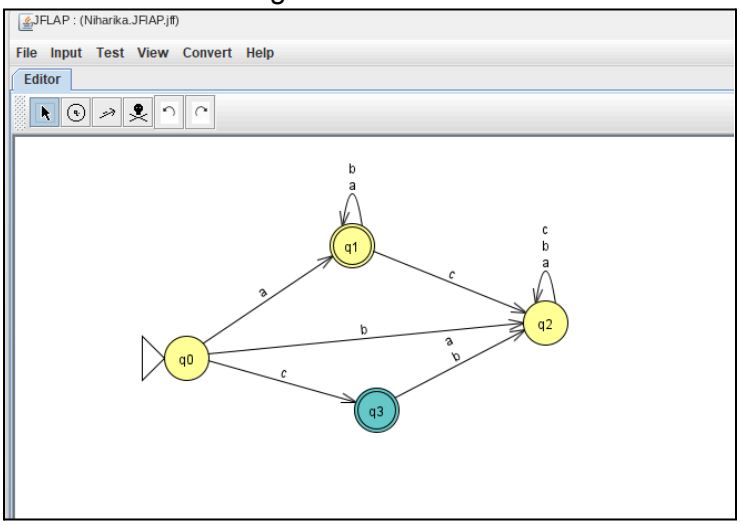
transition.
3. Create a single new final state and connect the original accepting states to it using epsilon transitions.
4. Combine multiple transitions between the same pair of states using the union operator.
5. Eliminate intermediate states one at a time and replace the affected paths with equivalent regular expressions.
6. Continue until only the new initial and final states remain.

When eliminating an intermediate state, the new transition expression is obtained using:
$R_i\square + R_i\square(R\square\square)^*R\square\square$
Where $R_i\square$ represents the expression entering the eliminated state, $R\square\square$ represents its self-loop, and $R\square\square$ represents the expression leaving it.

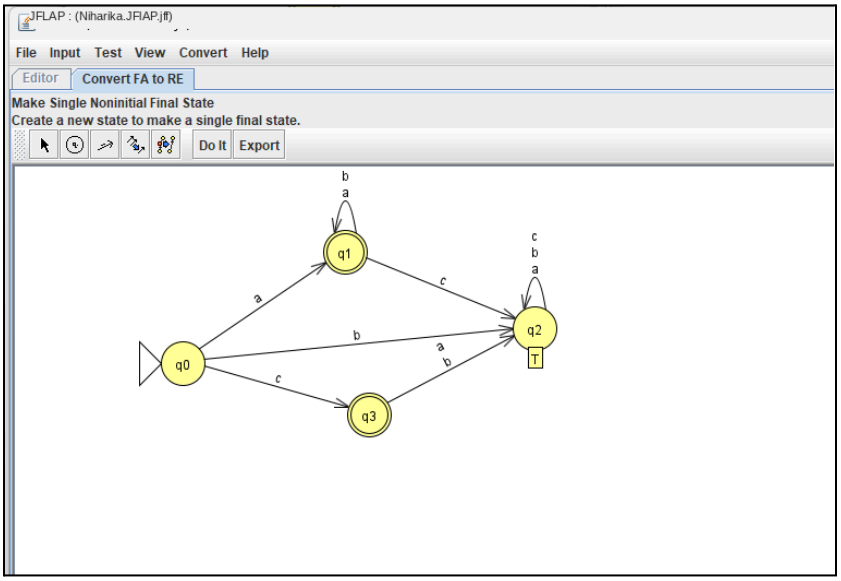
The final regular expression describes the language recognized by the original DFA.

Example:

Consider the following DFA

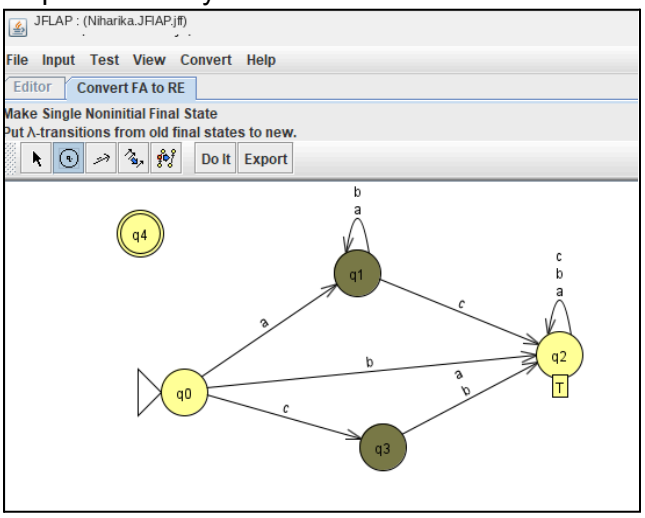


Step 1:

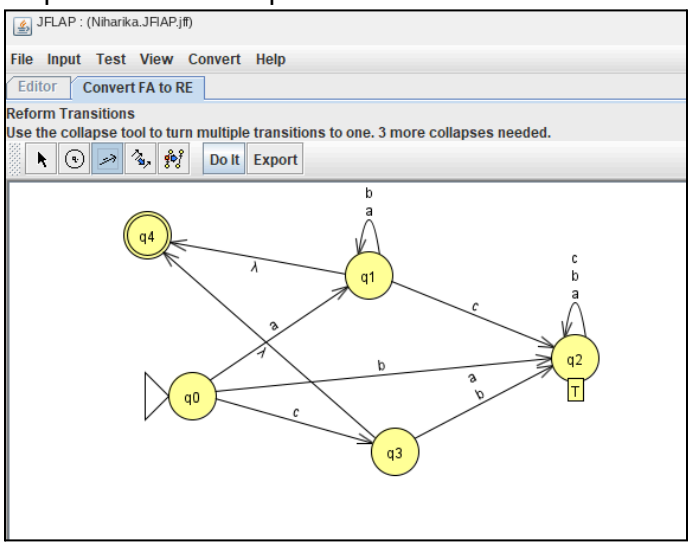


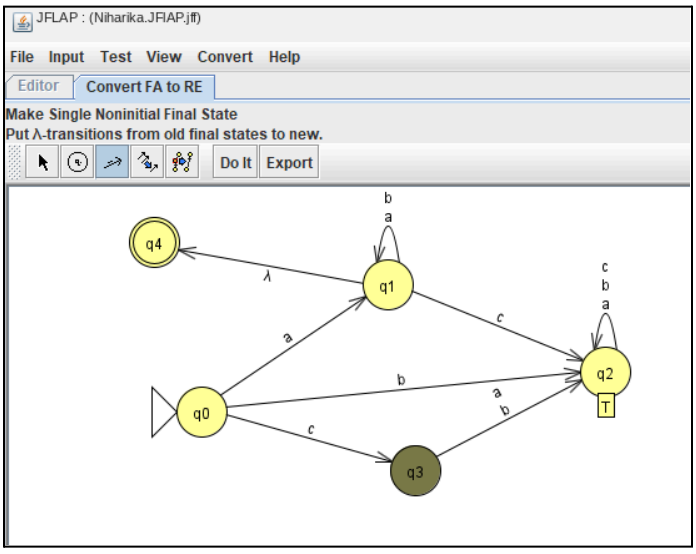
Niharika Pawar
D12C Roll no.44

Step 2: Manually create a new state

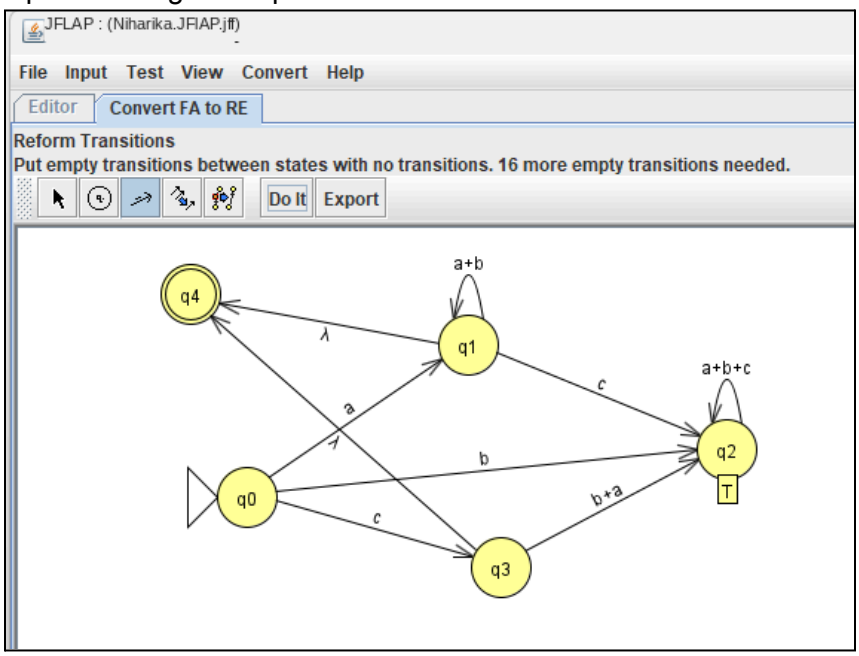


Step 3: Add each required λ-transition.

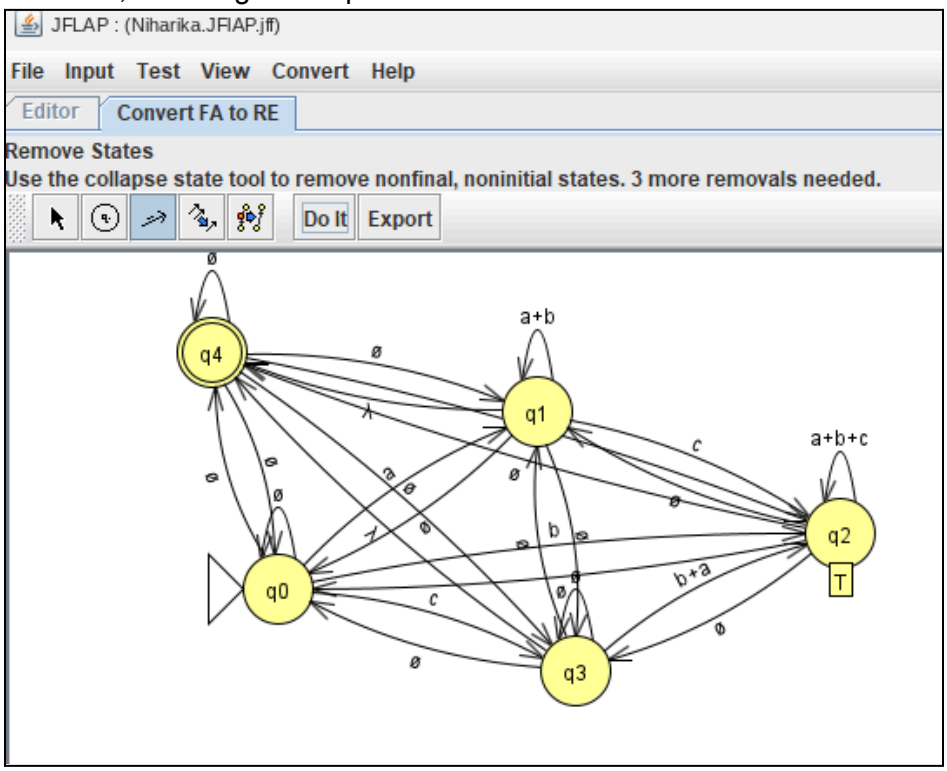




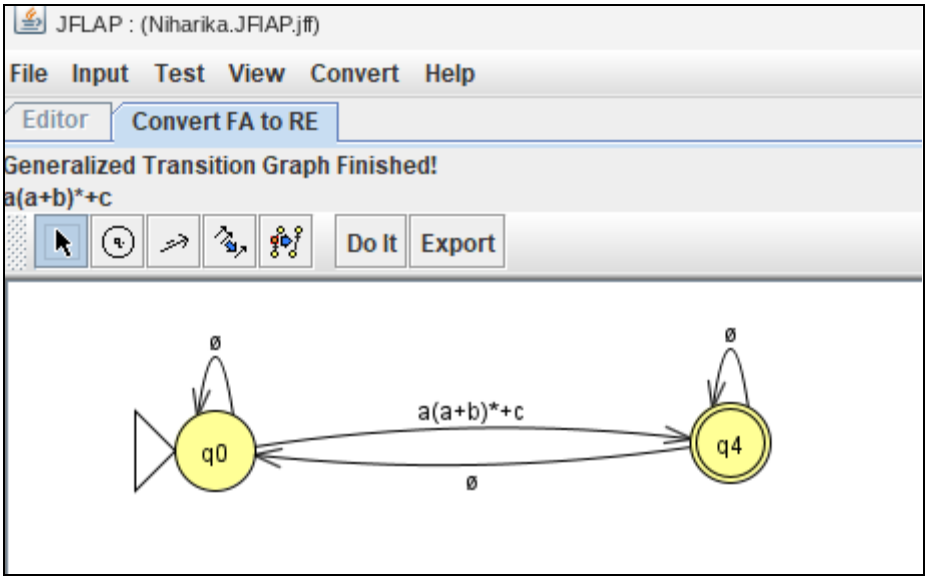
Step 4: The next step is to collapse parallel transitions, labeling the resulting transitions with the equivalent regular expressions. Here is the result.



Step 5: Add empty transitions between every pair of states that lack an explicit existing transition, including states paired with themselves.



Step 6: Finally, the collapse state tool provides a convenient means to eliminate states by replacing them with transitions labeled with the equivalent regular expression. The end result is shown here.



Therefore we have successfully converted DFA to RE

7) Combining Automata

Theory:

Combining Automata is a technique used to construct a new finite automaton from two or more existing automata. The resulting machine recognizes a language formed by applying operations such as union, intersection, complement, or difference to the original languages.

Regular languages possess a property known as closure, meaning the application of these operations produces another regular language.

Important Operations:

1. Union: The resulting automaton accepts a string if it belongs to at least one of the original languages.

$L = L_1 \cup L_2$

2. Intersection: The combined automaton accepts only those strings that are accepted by both original automata.

$L = L_1 \cap L_2$

3. Complement: The complement of a complete DFA is obtained by interchanging accepting and non-accepting states.

$L' = \Sigma^* - L$

4. Difference: This operation accepts strings belonging to the first language but not the second.

$L = L_1 - L_2$

Combining Procedure:

1. Construct the two automata representing the required languages.
2. Identify the operation to be applied.
3. For intersection using De Morgan's law, first obtain the complements of both languages.
4. Combine the complemented automata using union.
5. Convert the resulting NFA into a DFA if necessary.
6. Complement the resulting DFA to obtain the intersection.

De Morgan's law states:

$L_1 \cap L_2 = (L_1' \cup L_2')'$

Niharika Pawar
D12C Roll no.44

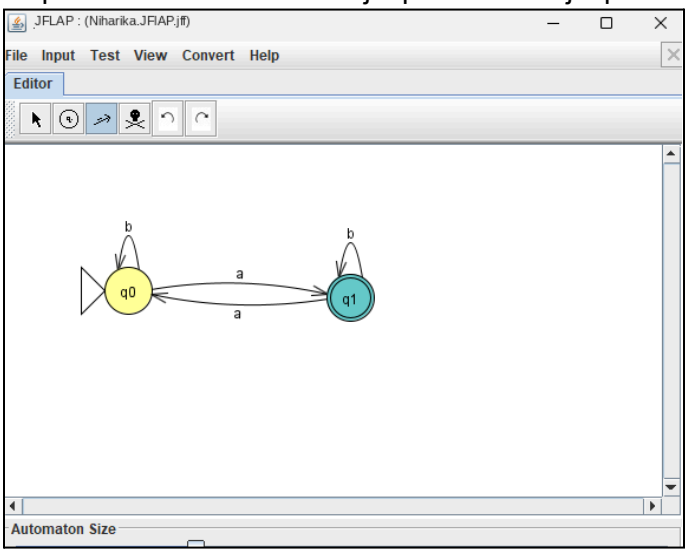
For example, two automata can be combined to recognize strings containing an odd number of a's while excluding strings containing 'ab' as a substring.

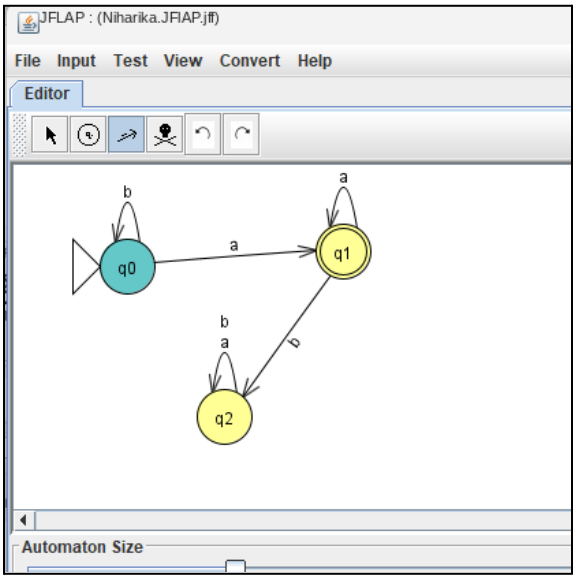
Combining automata is useful for developing complex language recognizers from simpler finite-state machines.

Example:

In this Exercise, we will use the “Combine Automata” feature of JFLAP to find the intersection of two DFA. For the alphabet Σ = {a, b}, consider the languages
L1 = {w ∈ Σ∗|w has an odd number of a’s}, and
L2 = {w ∈ Σ∗|w does not have ab as a substring}

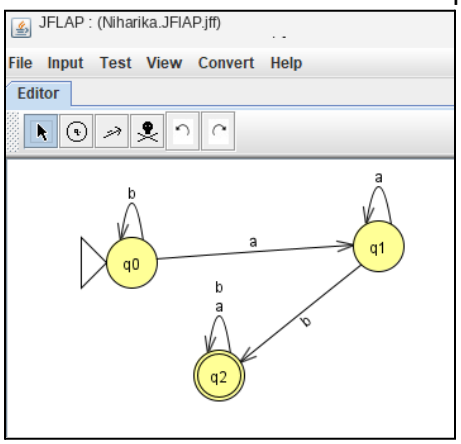
Step 1: Load the files odd-a.jflap and no-ab.jflap in two separate windows, as below

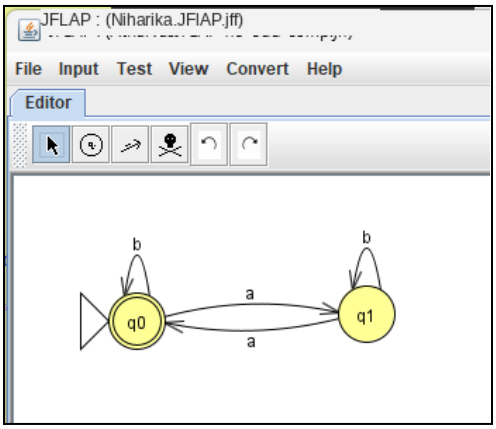




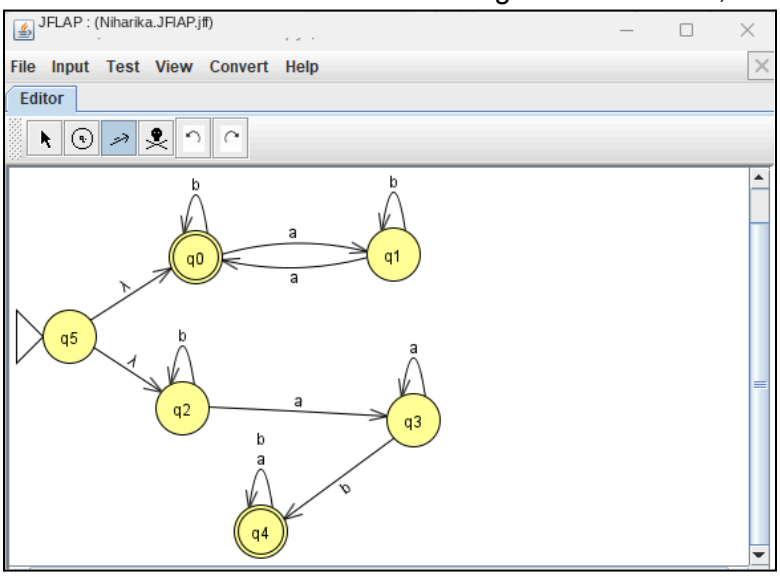
Step 2: Recall De Morgan’s Rule: L1 ∩ L2 = L1 ∪ L2. Hence L1 ∩ L2 = L1 ∪ L2. Hence to find the intersection of two DFA, we first find the complements of the DFA, then take their union, and finally again the complement.

Create two more DFA that are complements of those two above. These are shown below



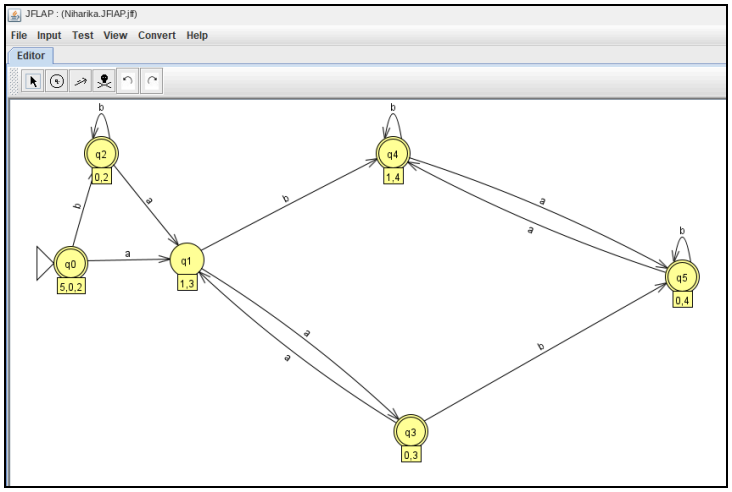


Step 3: In one of the windows of these two DFA, choose Convert from the menu and click on Combine Automata. A dialog box asks you to pick the second DFA. This opens a new window in which both DFA are drawn. Only one has an initial state.
To take the union of these the two, add a new initial state and add two λ-transitions from this new initial state to the two original initial states, as below

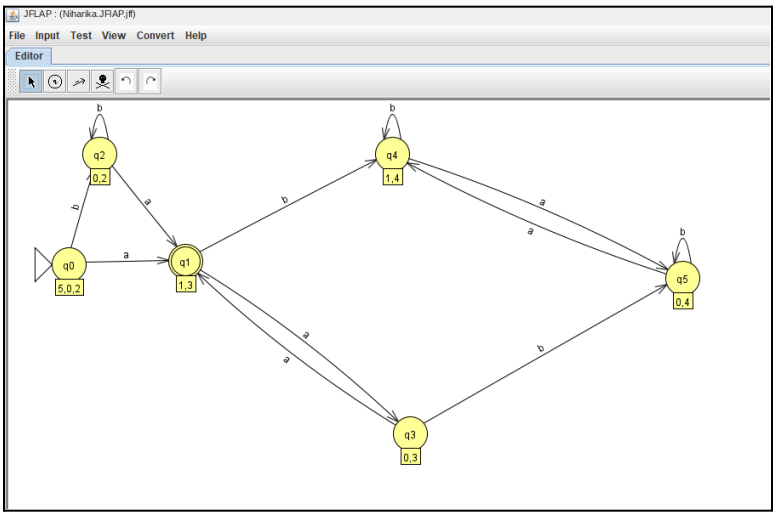


Step 4: Convert this NFA into a DFA.

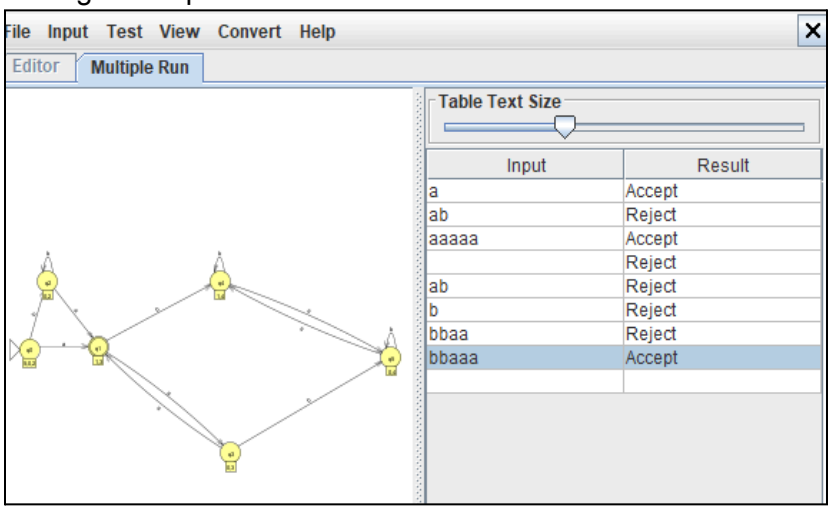
Niharika Pawar
D12C Roll no.44



Step 5: Find the complement of this last DFA.



Testing with Inputs:



Hence this DFA accepts all strings that have odd no. of a's and have no ab substring

8) Regular Expression to DFA conversion

Theory:

A Regular Expression can be converted into an equivalent DFA using a sequence of transformations. Since regular expressions and finite automata represent the same class of languages, any language described by a regular expression can also be recognized by a DFA.

The conversion is generally performed in two major stages: constructing an epsilon-NFA from the regular expression and converting that NFA into a DFA.

Stage 1: Regular Expression to NFA
Thompson's Construction is used to create an epsilon-NFA by dividing the expression into smaller components.

The construction follows these principles:

- For a single symbol, create two states connected by a transition labeled with that symbol.
- For union, construct alternative paths using epsilon transitions.
- For concatenation, connect the accepting state of one component to the starting state of the next component.
- For Kleene star, introduce epsilon transitions that allow zero or multiple repetitions of a pattern.

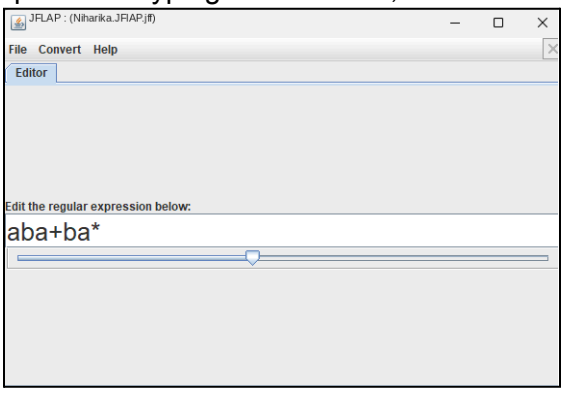
Stage 2: NFA to DFA
After constructing the NFA, the Subset Construction Method is applied.

1. Calculate the epsilon closure of the NFA's initial state.
2. Represent the resulting state set as the first DFA state.
3. Determine reachable state combinations for every input symbol.
4. Create new DFA states for each unique subset.
5. Continue until every transition has been processed.
6. Mark subsets containing an accepting NFA state as final states.

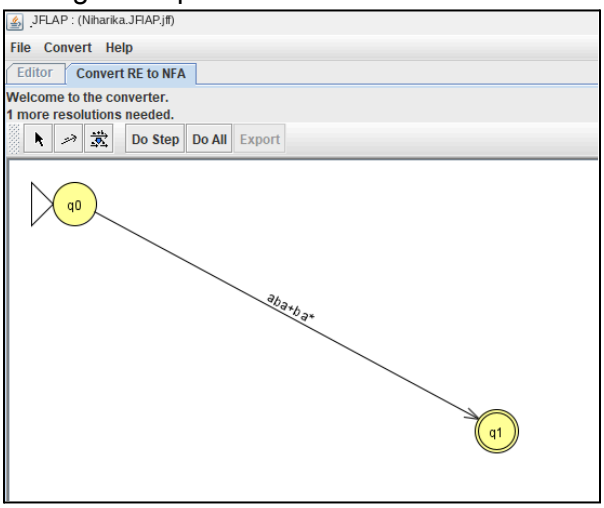
For example, the regular expression (a+b)*ab represents all strings over ending with 'ab'. Its equivalent DFA accepts precisely those strings.
This conversion is important in compiler construction, lexical analysis, and string pattern-matching applications.

Example:

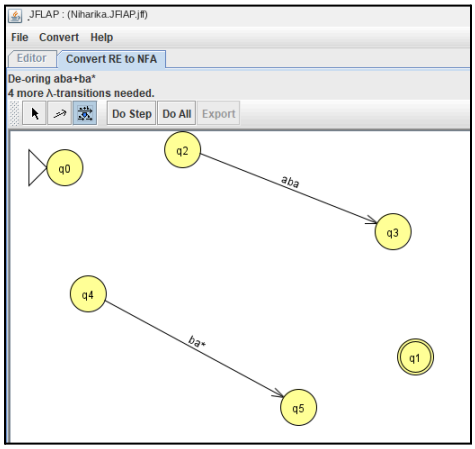
Convert the following Regular Expression to a DFA aba ∪ ba∗
Step 1: The regular expression can be entered into JFLAP by choosing the regular expression option and typing in aba + ba∗, since JFLAP uses + to represent the ∪.



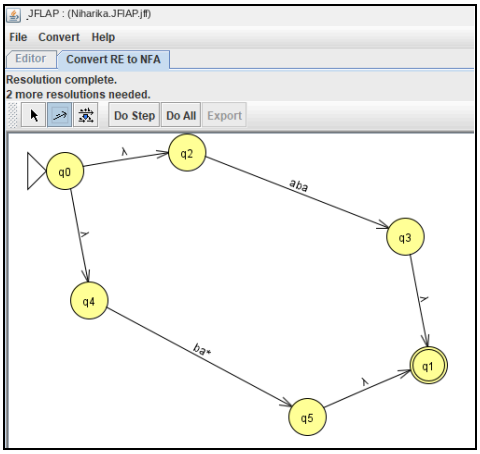
Step 2: Convert the regular expression to an NFA. This is done by breaking the regular expression down into its constituent parts.



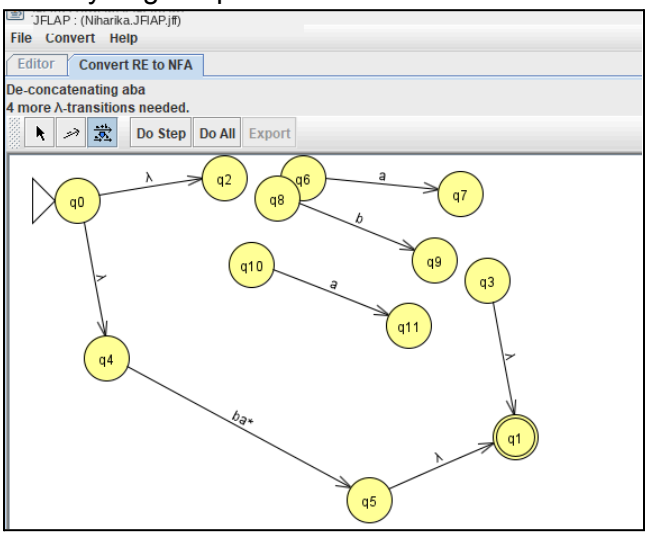
Step 3: Click the 'de-expressionify' transition button and then click the transition to get the two regular expressions that are separated by the union operator. So you get one transition of aba and one of ba



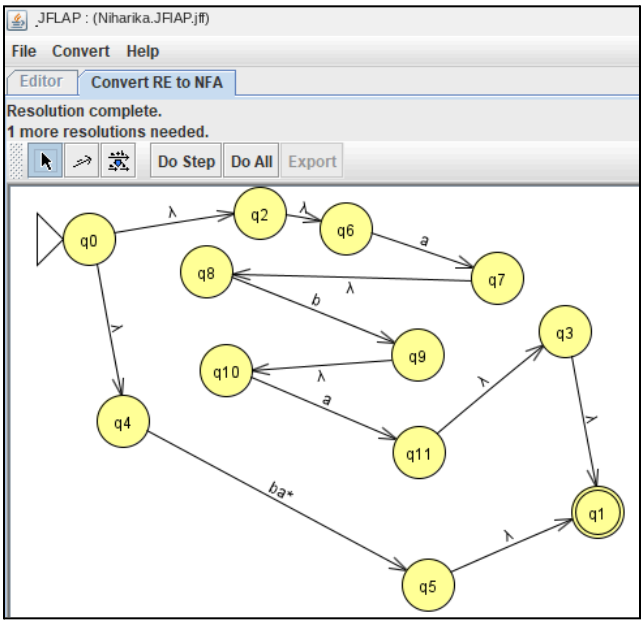
Step 4: Now to connect up these 'pieces' of the regular expression we will use epsilon transitions.



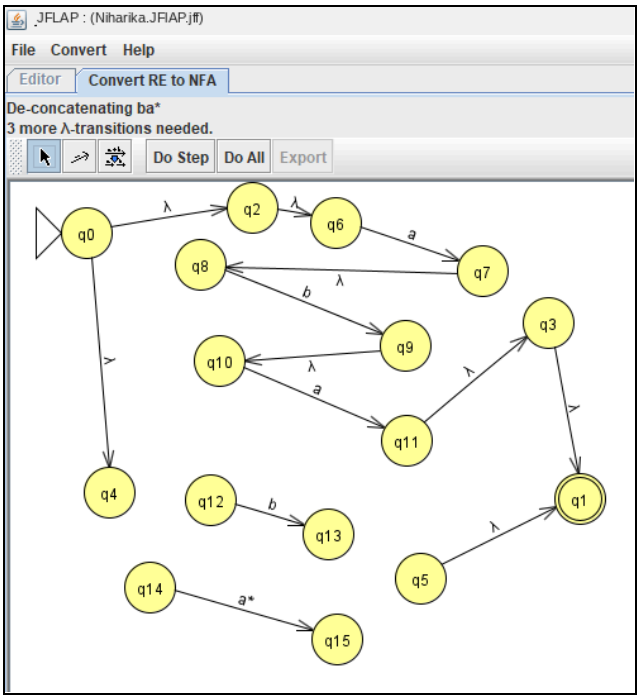
Step 5: In the next step we choose to break the expression aba. This is basically an a being con-catenated with a b being concatenated with an a. Upon clicking the De-expressionify button you get a picture that looks like.



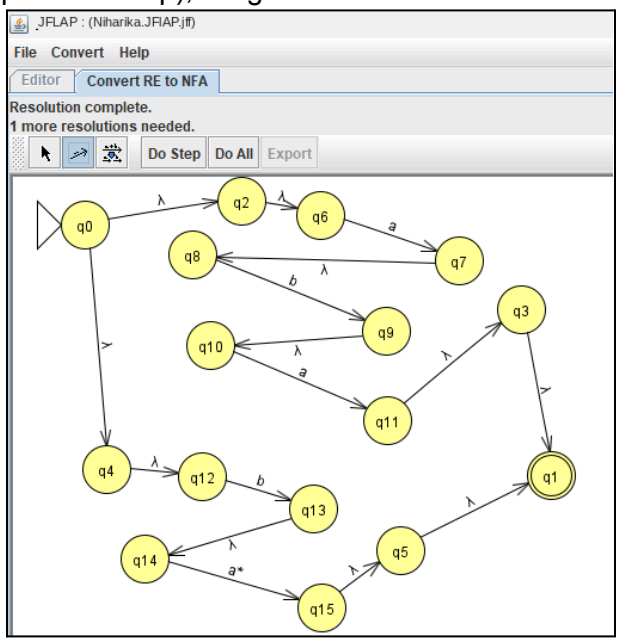
Step 6: Connect up the states via epsilon transitions. For concatenation you just need to transition from the final state of one to the start state of the next via an epsilon as can be seen here



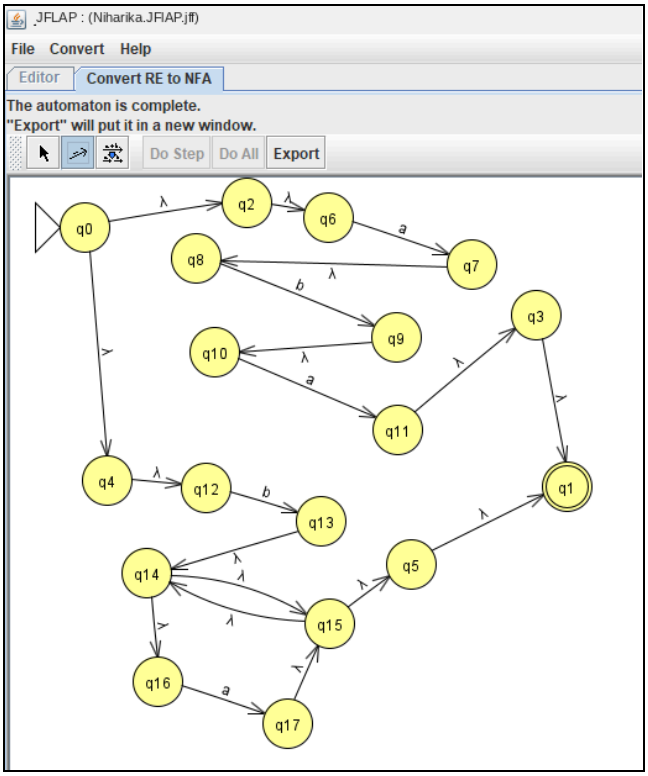
Step 7: Once aba has been broken up into its constituent NFA pieces, we turn our attention to ba∗ which upon de-expressionfication first looks like



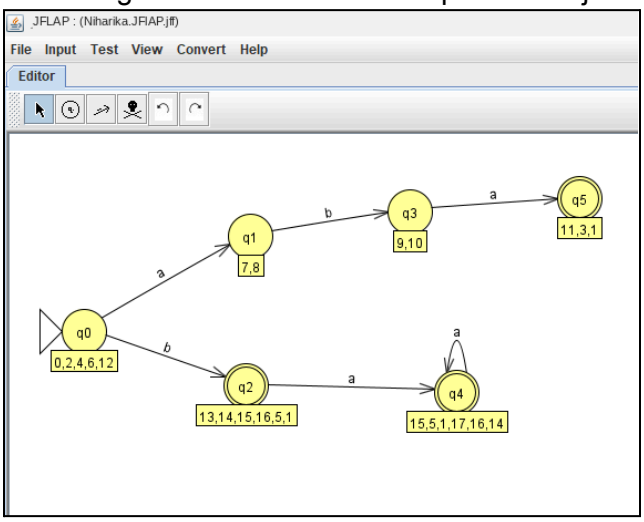
Step 8: And then upon connecting using epsilon transitions (similar to what we did in the previous step), we get



Step 9: In the final step we need to break up a∗. Upon clicking the de-expressionify button you get two more states that are connected by an epsilon transition. To create the effect of a ∗ operation, remember that ∗ means 0 or more instances being concatenated with each other. So, create an epsilon transition loop from q18 to q19. And then connect up q18 to q20 and q21 to q19 with epsilon transitions. By doing this, the q20 to q21 transition can be used multiple times.



Step 10: The final step would be to convert this now to a DFA. The convert menu has a convert to the DFA option which can be used. Refer to the NFA to DFA conversion help to get more insight into this. In this example we will just use the 'Complete' button to do this in 1 step.



9) Mealy Machine

Theory:

A Mealy Machine is a finite-state machine that generates output based on both its current state and the input symbol being processed. Unlike ordinary finite automata, which primarily determine acceptance or rejection, a Mealy machine produces output during transitions.

In this model, each transition is associated with an input-output pair. Whenever an input

is received, the machine changes its state and immediately generates the corresponding output.

Formal Definition:

A Mealy Machine is represented by the six-tuple:
$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$
Where:

- Q = Finite set of states.
- Σ = Input alphabet.
- Δ = Output alphabet.
- δ = State transition function, $\delta: Q \times \Sigma \rightarrow Q$.
- λ = Output function, $\lambda: Q \times \Sigma \rightarrow \Delta$.
- q_0 = Initial state.

Working Principle:

The machine begins at the initial state and reads an input character. Based on the current state and the received symbol, it selects the next state and produces an output.

Transitions are commonly labeled in the form:

Input / Output
For example, a Mealy Machine can be constructed to process binary strings by replacing the first '1' in each consecutive sequence of ones with '0'.
For the input:
0001001110
The corresponding output is:
0000000110
This is achieved by maintaining states that indicate whether the machine is processing zeros or a consecutive sequence of ones.

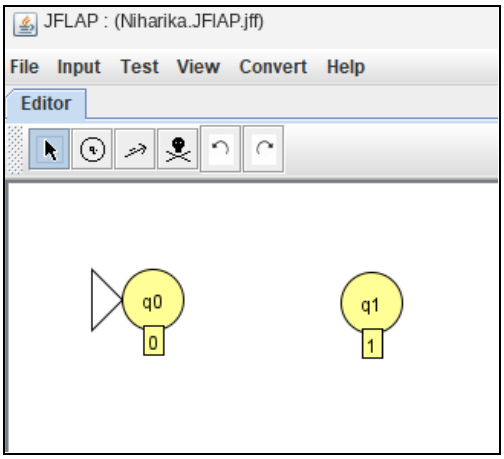
Applications:

Mealy Machines are used in sequence detectors, communication protocols, digital circuits, control systems, and input-output processing.
Their major advantage is that outputs can change immediately in response to an input without requiring an additional state transition.

Example:

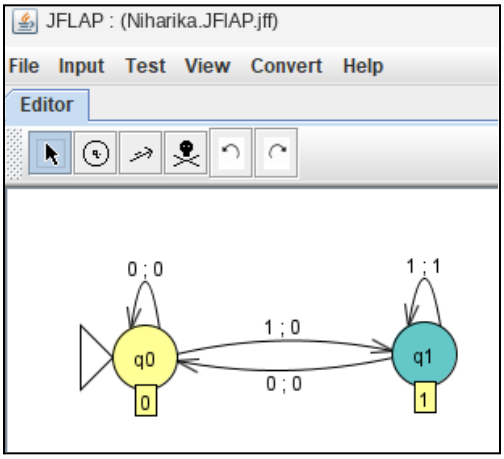
Construct a Mealy machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of “bit stuffing” may be used in data transmission and telecommunications for run--length coding to limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bit

Step 1:
Create a Mealy machine with an initial state. Set its label to a “0” to remember that a zero is read. Next, add a second state to remember that a “1” was read. Label this with a “1

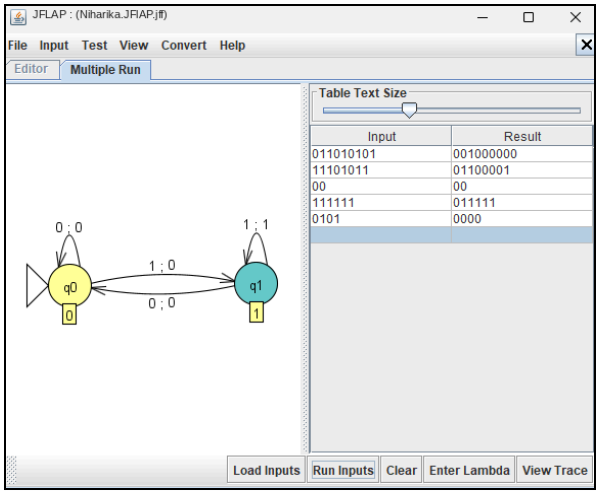


Step 2: Four transitions will be needed.

1. At q0, a 0 is read so a loopback to q0 is needed. Output a 0.
2. At q0, a 1 is read so a transition to q1 is needed. Output a 0 since this is the first 1 in a substring starting with a 1.
3. At q1, a 0 is read so a transition to q0 is needed. Output a 0.
4. At q1, a 1 read so a loopback to q1 is needed. Output a 1



Testing some inputs



10) Moore Machine

Theory:

A Moore Machine is a finite-state machine in which output depends entirely on the current state of the system. Each state has a predefined output value, which is produced whenever the machine enters that state.

Unlike a Mealy Machine, where outputs are associated with transitions, a Moore Machine associates output symbols directly with states.

Therefore, the output changes only when the machine moves from one state to another.

Formal Definition:

A Moore Machine is described by the six-tuple:
$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$
Where:

- Q = Finite set of states.
- Σ = Set of input symbols.
- Δ = Set of output symbols.
- δ = Transition function, $\delta: Q \times \Sigma \rightarrow Q$.
- λ = Output function, $\lambda: Q \rightarrow \Delta$.
- q_0 = Initial state.

Working Principle:

The machine begins at the initial state, which has an associated output. Each input symbol determines the next state, and the output corresponding to that state is produced.

Because the initial state also generates an output, the complete output sequence of a Moore Machine normally contains one more symbol than the input sequence.

For example, a Moore Machine can be designed to process a binary sequence by replacing the first occurrence of '1' in each continuous group of ones with '0'.

For the input:
0001001110
The processed output, excluding the initial state's output, is:
0000000110
This operation can be performed using three states representing the reading of zero, the first one, and the remaining consecutive ones.

Applications:

Moore Machines are used in digital controllers, sequential circuits, traffic signal systems, vending machines, and other state-based control applications.
Since the output depends only on the current state, Moore Machines generally provide stable and predictable output behavior.

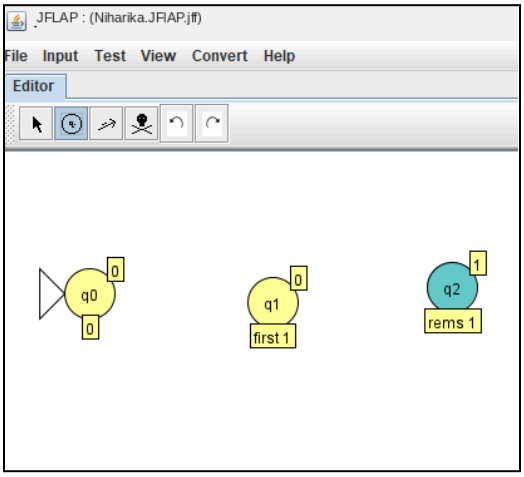
Example:

Construct a Moore machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of “bit stuffing” may be used in data transmission and telecommunications for run--length coding to limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bits.

Step 1:
Create a Moore machine with an initial state. Create three states as follows:

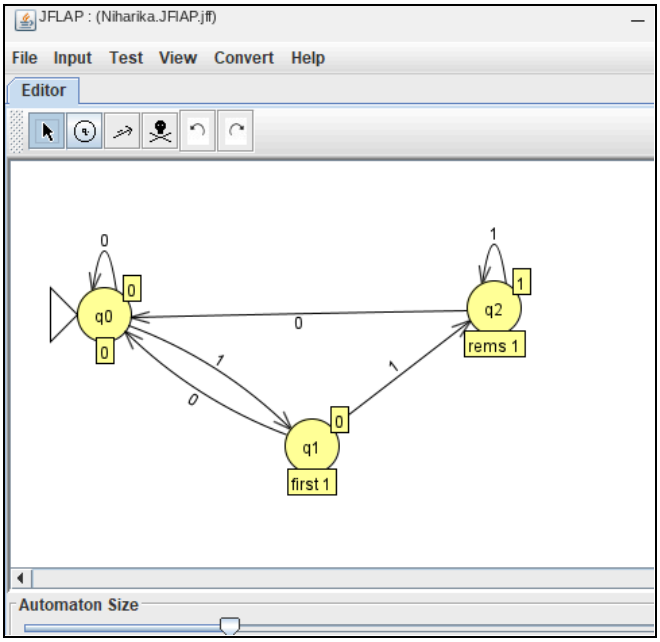
Niharika Pawar
D12C Roll no.44

1. q0 with an output of 0 to represent a 0 read.
2. q1 with an output of 0 to represent the first 1 read within a substring of 1s.
3. q2 with an output of 1 to represent the remaining 1's read after the first 1 was read.

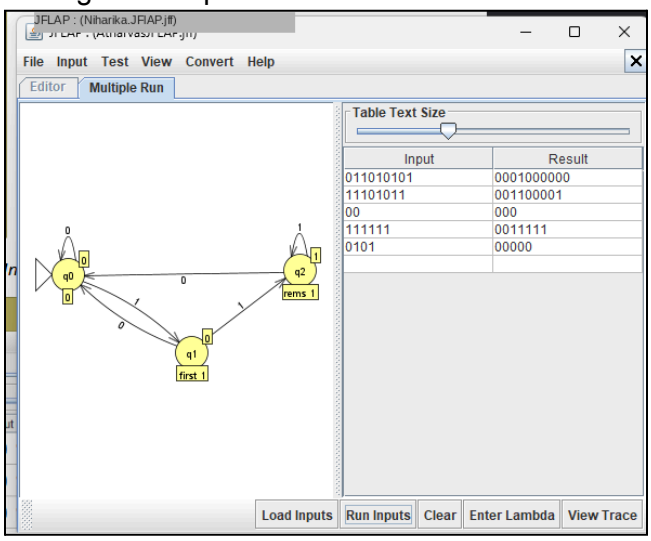


Step 2:
Add the transitions:

1. At q0 and an input of 0, stay at q0.
2. At q0 and an input of 1, go to q1.
3. At q1 and an input of 0, go back to q0.
4. At q1 and an input of 1, go to q2.
5. At q2 and an input of 0, go all the way back to q0.
6. At q2 and an input of 1, stay at q2.



Testing some inputs



CONCLUSION:

Thus, various concepts of Automata Theory were successfully studied and implemented using JFLAP software. The activities involved designing and testing DFA and NFA models, performing NFA-to-DFA conversion, minimizing deterministic automata, and converting between finite automata, regular grammars, and regular expressions. Additionally, combining automata and implementing Mealy and Moore machines helped demonstrate different approaches to language recognition and output generation. These experiments provided practical understanding of state transitions, regular languages, and finite-state computation.